

main

May 2, 2025

```
[1]: # Define the verbose variable
verbose = False

# Check if we're running in a Jupyter Notebook (not compiled script)
try:
    __IPYTHON__
    verbose = True # Set verbose to True if running in a Jupyter Notebook
except NameError:
    pass # Do nothing if we're not in a notebook
```

```
[2]: # All the imports

import torch
from torch.utils.data import TensorDataset
from torch import nn
import matplotlib.pyplot as plt

from the_ml.load_dataset import labelList, get_data
from the_ml.preprocess import preprocess_dataset
from the_ml import bigmodel

from ml_util.eeg_util import plotEEG, plotTimeFreqEEG
from ml_util.checkpoint import checkpoint
from ml_util.data_module import DataModule
from ml_util.trainer import GoodClassificationModel, GoodTrainer
from ml_util.logger import WandBLogger, print_stats
from ml_util.reporter import getTestResults
from ml_util.plot import plot_confusion_matrix

# To make it reproducible
torch.manual_seed(42)
```

```
/Users/ulyssedurand/git/eeg-ml/EEG-classification/venv/lib/python3.10/site-
packages/tqdm/auto.py:21: TqdmWarning: IProgress not found. Please update
jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user\_install.html
    from .autonotebook import tqdm as notebook_tqdm
```

```
[2]: <torch._C.Generator at 0x10807e510>
```

```
[3]: print("Loading raw dataset")
X_raw, y = checkpoint(lambda: get_data(verbose=True), "raw_input")
print(f"Raw input dataset size: {X_raw.shape}")
```

Loading raw dataset

Raw input dataset size: torch.Size([480, 32, 3200])

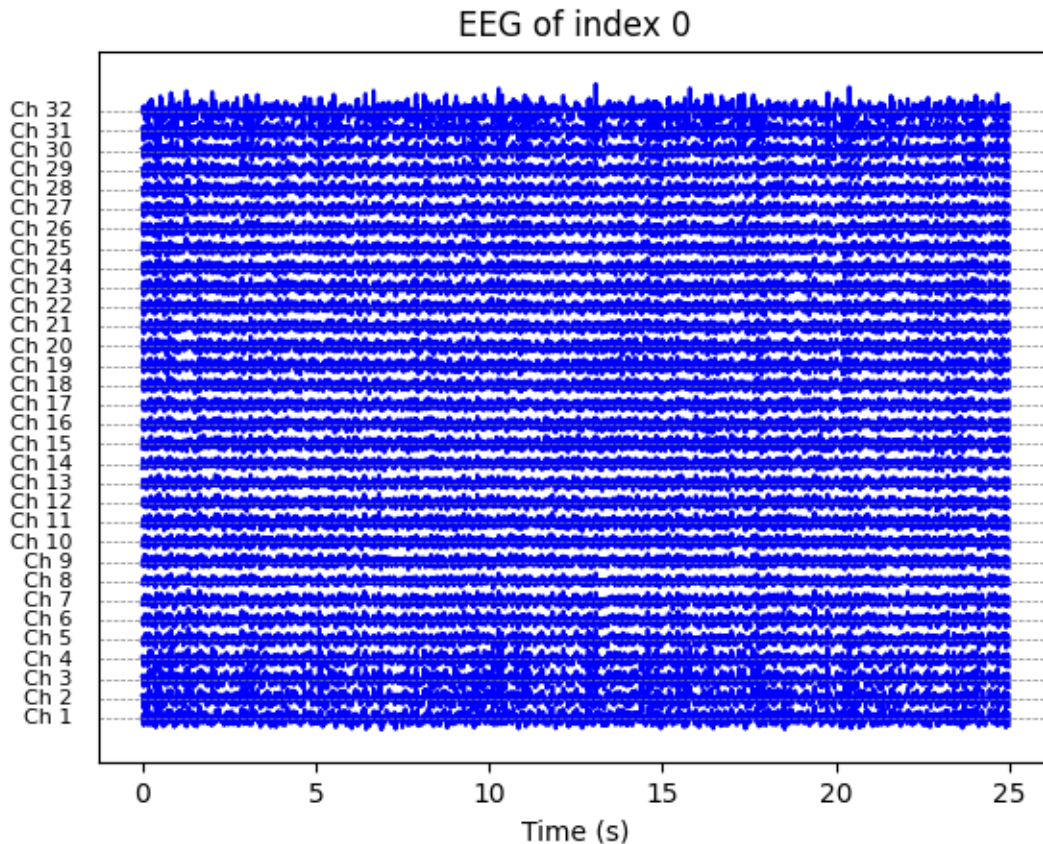
1 The SAM40 Dataset

It consists of electroencephalograms (EEG) of people while they perform tasks.

You can find the dataset [here](#)

We extract the dataset to obtain 480 samples (120 for each task among Stroop, Relax, Mirror_Image, and Arithmetic). Each sample has the record of the 32 electrodes of the EEG over 25 seconds at 128Hz (3200 points)

```
[4]: if verbose:
    plotEEG(X_raw[0], title="index 0")
    print(labelList[y[0]])
```



Arithmetic

1.1 Preprocessing

We don't want to feed the raw inputs to our model, we will extract features from it.

The approach used was to use a time-frequency spectrogram with the alpha, beta, gamma, delta and theta bands (those are intervals in the frequency domain).

After the preprocessing step we have a 32 channels 5x101 image for each sample. 5 corresponds to the 5 frequency intervals and 101 are the time intervals.

```
[5]: hyperparams={"bands_param": "five"} # hop time in seconds

# Applies preprocessing
print("Preprocessing input data")
X = checkpoint(lambda : preprocess_dataset(X_raw, hyperparams["bands_param"],
    verbose=True), "preprocessed")
print(f"Preprocessed input dataset shape: {X.shape}")

# Stores in a TensorDataset
dataset = DataModule(X, y, val_part=0.15, test_part=0.15)

# Uncomment to reduce the size of the dataset
# dataset = dataset.get_part(0.1)
```

Preprocessing input data

Preprocessed input dataset shape: torch.Size([480, 32, 5, 101])

```
[6]: if verbose:
    for label in range(4):
        sampleId = torch.where(y==label)[0][0]
        plt.figure()
        fig, axes = plotTimeFreqEEG(X[sampleId])
        #plt.imshow(X[sampleId][0])
        fig.suptitle(f"Sample {sampleId}, with label {labelList[y[sampleId]]}")
        plt.show()
```

<Figure size 640x480 with 0 Axes>

Sample 0, with label Arithmetic



<Figure size 640x480 with 0 Axes>

Sample 120, with label Mirror_image



<Figure size 640x480 with 0 Axes>

Sample 240, with label Relax



<Figure size 640x480 with 0 Axes>

Sample 360, with label Stroop



1.2 The model

Here are the layers of the model: - BatchNorm2d - Conv2d
32 to 128 channels, kernel_size of 5, stride 1, padding 2 - Dropout2d
0.3 probability of channel dropout - BatchNorm2d - ReLU - Conv2d
128 to 64 channels, kernel_size of 5, stride 1, padding 2 - Dropout2d
0.2 probability of channel dropout - BatchNorm2d - ReLU - Conv2d
64 to 32 channels, kernel_size of 5, stride 1, padding 2 - Dropout2d
0.2 probability of channel dropout - BatchNorm2d - ReLU - Flatten - Fully-Connected 321015 to
128 layers - Dropout
0.5 probability of neuron dropout - BatchNorm - ReLU - Fully-Connected 128 to 4 layers - SoftMax
loss

1.3 The training

- Adam optimizer was used
- L2 regularization added to the loss
- Every epoch, the learning rate is reduced by 1%

```
[7]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")  
print("GPU" if torch.cuda.is_available() else "CPU")
```

```

hyperparams |= {
    'lr': 1e-4,
    'weight_decay': 2e-2,
    'batch_size': 512
}
model = bigmodel.model(X[0], hyperparams)
loss_fn = nn.CrossEntropyLoss()

classification_model = GoodClassificationModel(model, loss_fn, hyperparams,
↪device, 4)
trainer = GoodTrainer()

def babysitter(goodModel):
    goodModel.hyperparameters['lr'] = 0.99 * goodModel.hyperparameters['lr']

trainer.add_babysitter(babysitter)
trainer.add_logger(lambda res: print_stats(res, print_every=100))
trainer.add_logger(WandBLogger(hyperparams, labelList, model))

```

CPU

wandb: Currently logged in as: **ulyssedurand**
(ulyssedurand-ens-de-lyon) to <https://api.wandb.ai>. Use
`wandb login --relogin` to force relogin

wandb: Tracking run with wandb version 0.19.10

wandb: Run data is saved locally in
/Users/ulyssedurand/git/eeg-ml/EEG-

classification/wandb/run-20250502_181443-blm26166

wandb: Run `wandb offline` to turn off syncing.

wandb: Syncing run **gentle-star-208**

wandb: View project at ↪
<https://wandb.ai/ulyssedurand-ens-de-lyon/EEG%20Classification>

wandb: View run at ↪
<https://wandb.ai/ulyssedurand-ens-de-lyon/EEG%20Classification/runs/blm26166>

```
[8]: trainer.train(classification_model, dataset, 10)
```

```

[9]: res = (getTestResults(dataset, classification_model))
print(f" Test accuracy: {res['test_acc']}")
test_preds, test_labels = res["test_confusion"]
plot_confusion_matrix(test_preds, test_labels, labelList)
plt.plot()

```

Test accuracy: 0.25

[9]: []

